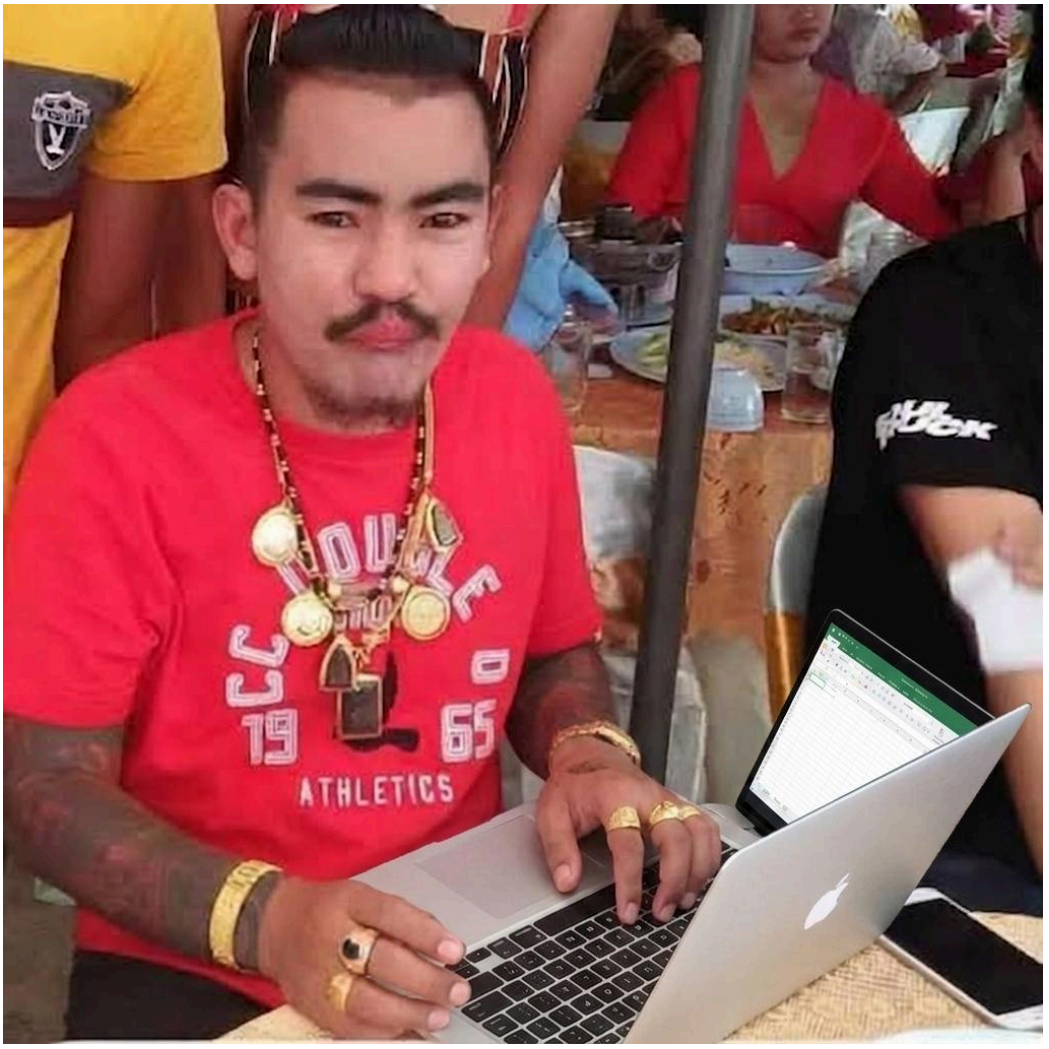
Course: Individual Software Development Process' 2026

Software Requirement Specification

By ISANPOWER888 (ISP888)

Chosen Irl Challenge: Project B: Lab Workflow & Request Management



Theewasu Aekthong	6810545701
Kantee Laibuddee	6710545440
Tanon Likhittaphong	6710545547
Inthat Niramarn	6810545999

Table of Contents

1. Target Users

- **Lab Member :** Submit requests (e.g., request equipment/space, order consumables, report issues, request access permissions), track the status of their own requests.
- **Teaching Assistant :** Receive and process incoming requests, update status, assign or route requests to the right person.
- **Lecturer / Lab Manager :** Approve requests that require authorisation (e.g., high-cost equipment, special access), view an overview and prioritise requests across the lab, view resource-usage reports.
- **System Administrator:** Manage user accounts and access permissions, maintain system configuration (request categories, statuses, etc.).
- **External Visitor / Industry Collaborator:** A non-member of the lab who requests access to the lab, categorised as either (a) **Open House / General Visit** submitted directly via a public request form without an account, or (b) **Industry Collaboration** submitted on their behalf by a TA/Lab Manager after external coordination (email/phone), since these requests are more formal and often require approval. Both types can track status via a tracking code/link without needing a full account.

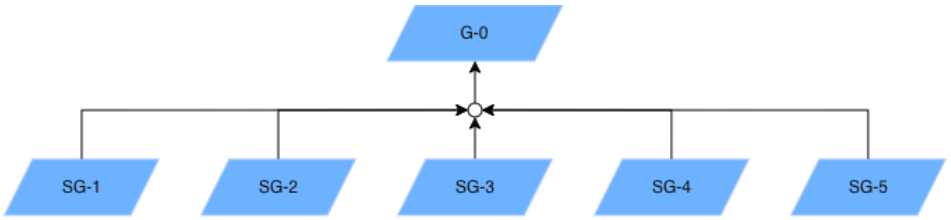
2. The System Goal

G-0: Make lab workflows and requests visible, trackable, and reliable.

The vase lab supports multiple activities (student projects, research, industry collaborations, outreach events). Day-to-day workflows — experiment setup requests, equipment maintenance, access permissions, consumables ordering, and internal tickets (e.g., "fix this script / calibrate sensor") — are currently handled via email threads, chat messages, and ad-hoc spreadsheets. As a result, important requests get lost, priorities are unclear, follow-up is hard to track, and new lab members have no central place to see what's going on or how to request support.

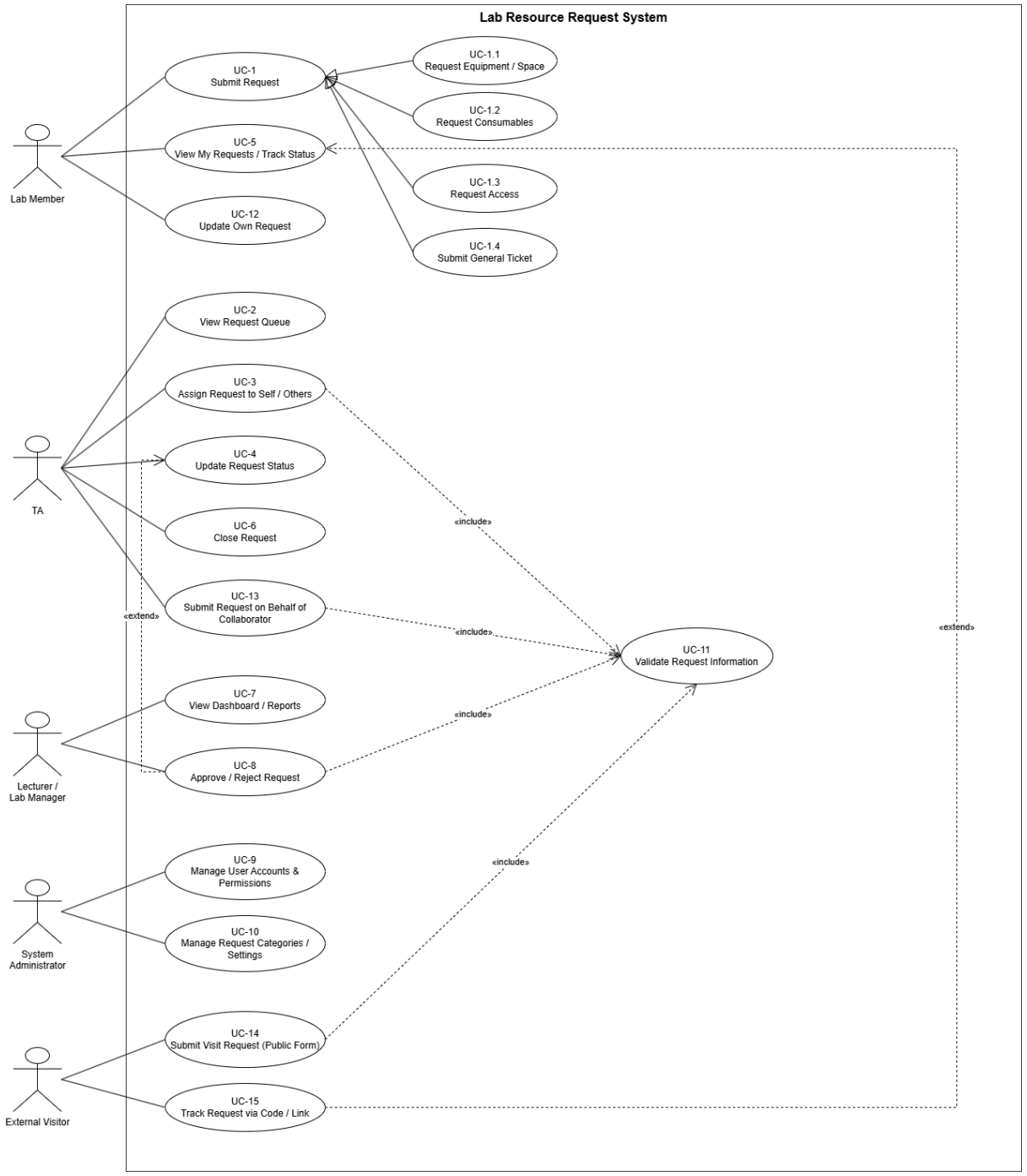
For this iteration, the system scope is limited to **request submission, status tracking, and request handling by authorised staff** for the main request types (equipment/space, consumables, access, general tickets). External/visitor requests (Open House and Industry Collaboration) are included within the "access" request type, with submission via two channels: a public self-service form (no login required) or staff-entry on behalf of the requester. Calendar/booking integration, automated notifications via Line/email, and in-depth analytics reporting are explicitly out of scope for this iteration and will be addressed in later iterations.

3. Goal Refinement via KAOS:



- **Sub-Goal 1 (SG-1)**
  Description: Achieve: Allow lab members to submit and categorise requests through a central system.
  Rationale: Requests are currently scattered across email, chat, and spreadsheets, causing them to be lost or hard to track.
- **Sub-Goal 2 (SG-2)**
  Description: Maintain: Complete and structured information for every request (type, requester, priority, status, timestamps).
  Rationale: Complete information allows staff to understand and act on requests correctly.
- **Sub-Goal 3 (SG-3)**
  Description: Achieve: Allow authorised staff (TA/Lab Manager) to review, assign, and update the status of requests.
  Rationale: Clear ownership is needed to ensure requests are handled and closed.
- **Sub-Goal 4 (SG-4)**
  Description: Avoid: Requests being lost or left unaddressed without visibility.
  Rationale: This is the core problem stated in the Problem Context important requests currently get lost due to the lack of a central place to track them.
- **Sub-Goal 5 (SG-5)**
  Description: Maintain: A traceable history of requests and status changes.
  Rationale: Supports auditing, resource-usage analysis, and gives new lab members visibility into past work.
- **Sub-Goal 6 (SG-6)**
  Description: Achieve: Allow external visitors and industry collaborators to submit lab-access requests without requiring a full system account, while allowing staff to submit formal collaboration requests on their behalf.
  Rationale: The lab serves external stakeholders (open house visitors, industry partners) who fall outside the internal user base but still need a trackable channel into the same request system.

4. Use Case Diagram



5. Use Stories

User Story ID	Description	Acceptance Criteria
US-1	As a lab member, I want to submit a request with a category and description so that TAs/lecturers can see and act on it.	Required fields (type, description, priority) must be filled. A valid request is saved with status "Pending" and appears in the queue.
US-2	As a lab member, I want to view the status of my submitted requests so that I know if action is being taken.	The requester can see a list of their own requests with current status and last-updated time.
US-3	As a TA, I want to view all pending requests in one place so that nothing gets lost or forgotten.	The queue shows all non-closed requests, sortable/filterable by type, priority, and status.
US-4	As a TA, I want to assign a request to myself or update its status so that progress is tracked.	Only authorised users (TA/Lab Manager) can change status. The requester sees the updated status.
US-5	As a lab manager, I want to approve or reject requests that need authorisation so that resource use stays controlled.	Requests marked "needs approval" cannot proceed until an authorised approver acts on them.
US-6	As a lab manager, I want to see a history/report of past requests so that I can understand resource usage over time.	Closed requests remain viewable with full history (who, what, when, status changes).
US-7	As an external visitor, I want to submit an open-house/lab-visit request through a public form without creating an account, so that I can request access easily.	Public form requires name, contact, organisation (optional), visit purpose, preferred date. On submit, systems generate a unique tracking code and show it to the requester.
US-8	As a TA, I want to submit a formal collaboration request on behalf of an industry partner, so that official partnerships are properly logged and routed for approval.	TA can create a request marked "External Industry Collaboration" with collaborator details; request is automatically flagged "needs approval" for Lecturer/Lab Manager.
US-9	As a registered user, I want to log in to the system so that I can access functions according to my role.	Valid registered users must provide valid login credentials. The system authenticates the user and grants access according to their assigned role. Invalid credentials are rejected and access is denied.

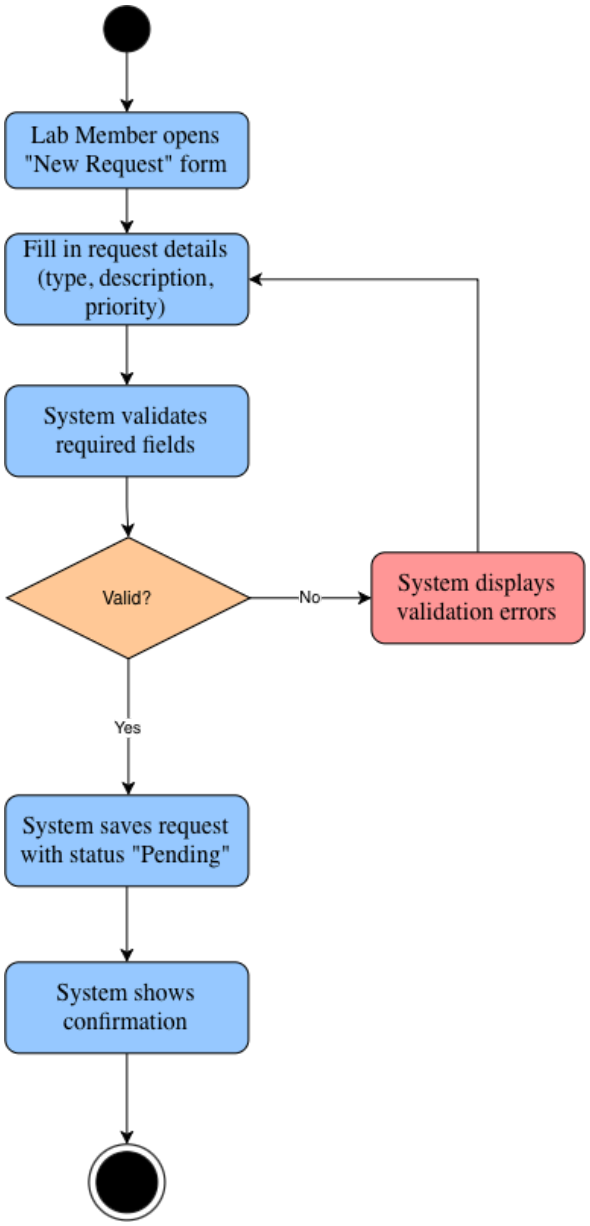
6. User Requirement Specification

ID	User Requirement
URS-1	Lab members shall be able to submit a request specifying type, description, and priority.
URS-2	Lab members shall be able to view the status and history of their own submitted requests.
URS-3	TAs shall be able to view, filter, and sort all open requests in a central queue.
URS-4	Authorised staff (TA/Lab Manager) shall be able to assign, update the status of, and close requests.
URS-5	The system shall allow designated requests to require approval from the Lab Manager before proceeding.
URS-6	The system shall maintain a traceable record of each request's status changes and timestamps.
URS-7	The System Administrator shall be able to manage user accounts and access permissions.
URS-8	The system shall allow external visitors to submit a lab-visit request via a public form without authentication.
URS-9	The system shall allow TAs to create a request on behalf of an external industry collaborator.
URS-10	The system shall provide external requesters with a tracking code or link to check their request status without logging in.
URS-11	Registered users can log in.
URS-12	Guest users can access the public request form without authentication.
URS-13	The system provides role-based access for authenticated users.

7. Activity Diagram

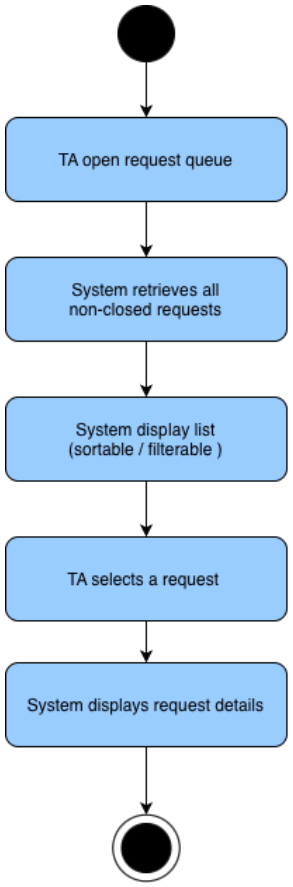
AD-1: Lab Member Submits a Request

This diagram captures the request-submission flow, including field validation, used to derive SRS-3, SRS-4, and SRS-5.



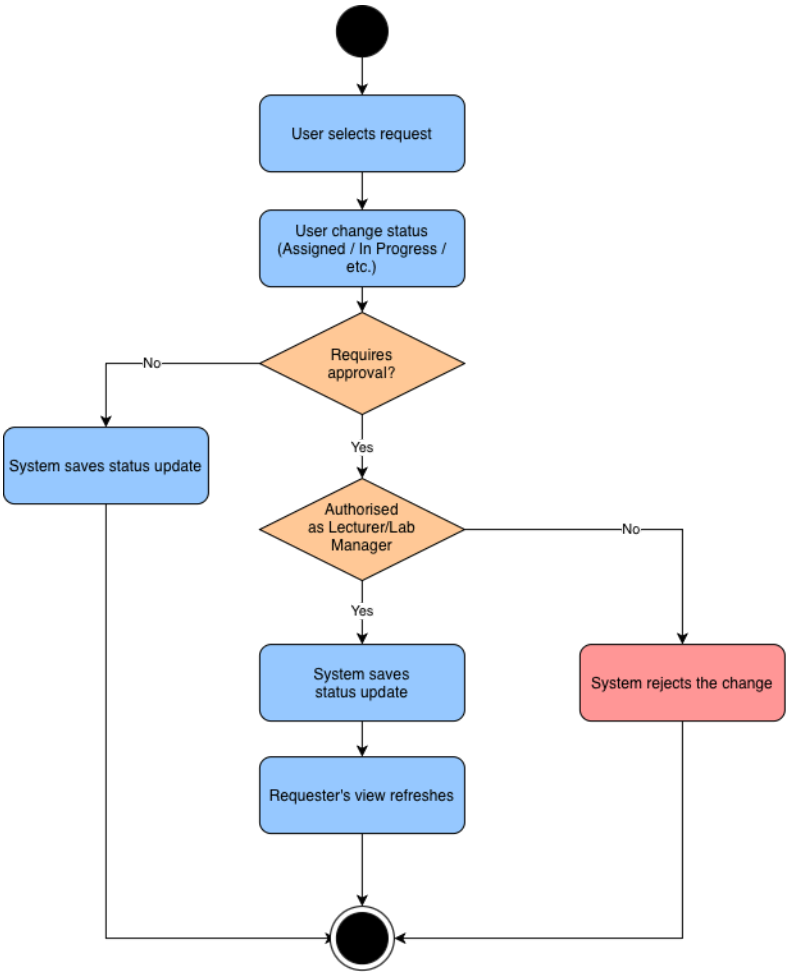
AD-2: TA Views Request Queue

This diagram traces the request-browsing flow used by TAs to derive SRS-1 and SRS-2.



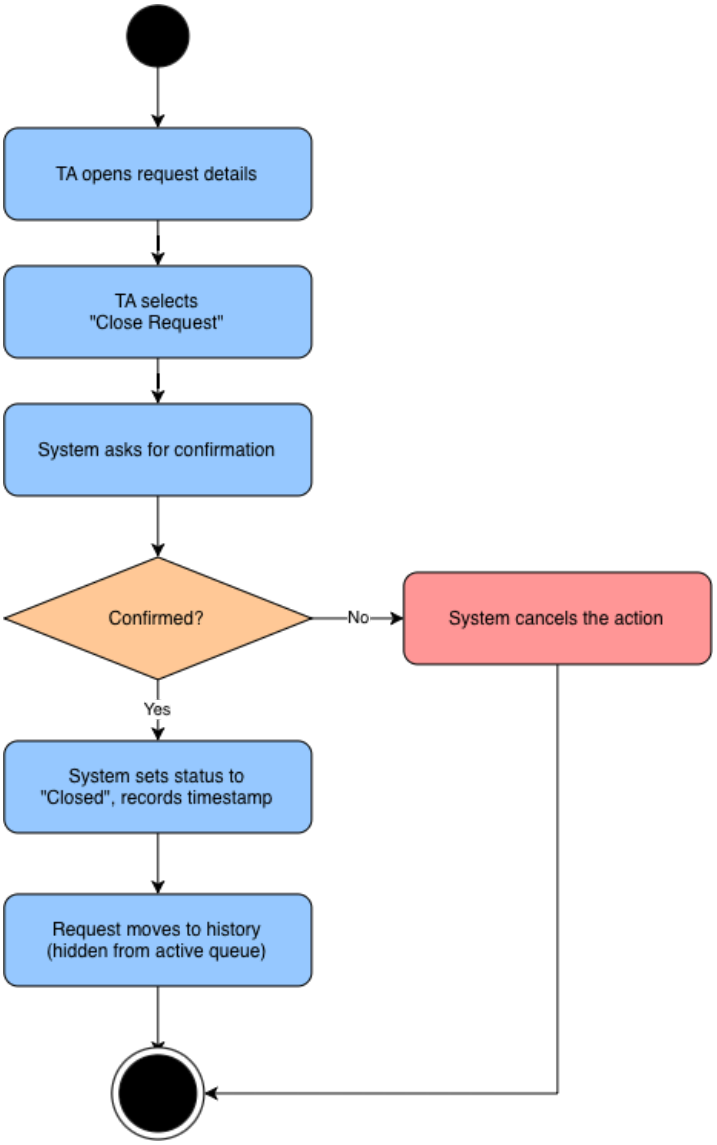
AD-3: Authorised Staff Updates Request Status (with approval check)

This diagram captures the authorisation check and status-update flow (including the approval path) used to derive SRS-6 and SRS-7.



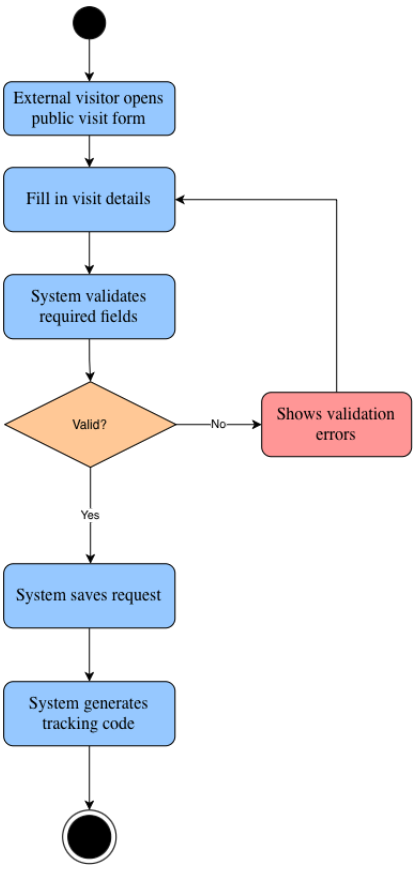
AD-4: TA Closes a Request

This diagram captures the confirmation and archive/close flow used to derive SRS-8.



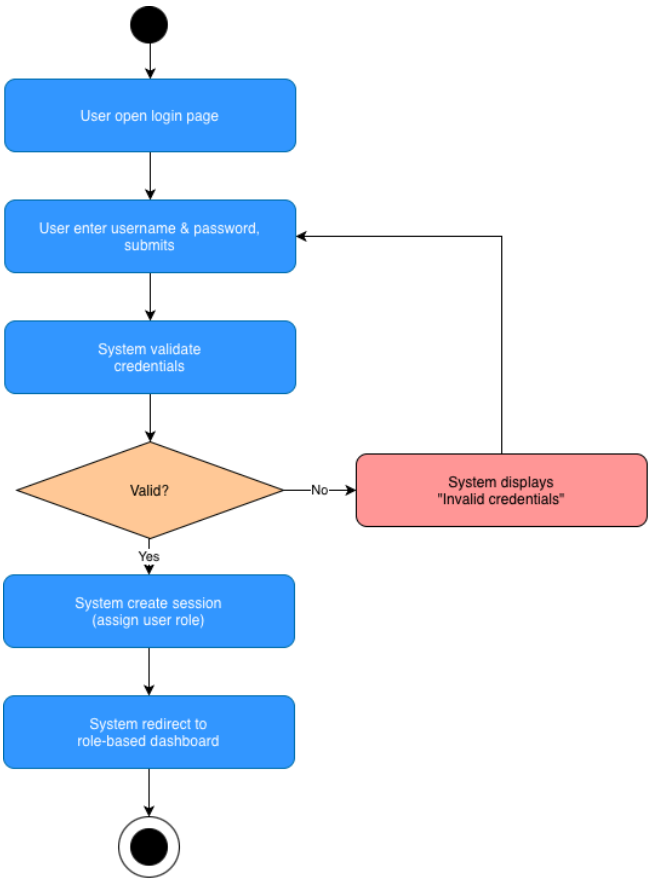
AD-5: External Visitor Submits Open House Request

This diagram captures the public, unauthenticated request-submission flow used by external visitors (open house / general public), including field validation and tracking-code generation, used to derive SRS-9, SRS-10, and SRS-5.



AD-6: User Logs in

This diagram captures the authentication check and role-based redirect flow used to derive SRS-13, SRS-14, and SRS-15.



8. System Requirement Specification

ID	System Requirement
SRS-1	The system shall display only non-closed requests in the active request queue.
SRS-2	The system shall display the type, requester, priority, and status of each request in the queue and in the detail view.
SRS-3	The system shall allow only authorised staff (TA/Lab Manager) to assign, update the status of, or close a request.
SRS-4	The system shall require a request type, description, and priority before saving a new request.
SRS-5	The system shall display a validation message and shall not save the request when required data is missing or invalid.
SRS-6	The system shall store each saved request with a status of Pending, Assigned, In Progress, Approved, Rejected, or Closed.
SRS-7	The system shall require Lecturer/Lab Manager approval before a

	request flagged "needs approval" can move past the Pending status, and shall refresh the requester's view after any saved status change.
SRS-8	The system shall remove closed requests from the active queue while retaining them in a searchable history for reporting.
SRS-9	The system shall allow submission of a lab-visit request via a public, unauthenticated form, capturing name, contact, organisation (optional), purpose, and preferred date.
SRS-10	The system shall generate a unique tracking code for each externally submitted request and display it to the requester upon submission.
SRS-11	The system shall allow an external requester to view their request status by entering their tracking code, without requiring login.
SRS-12	The system shall allow a TA to create a request record on behalf of an external industry collaborator, tagged as "External — Industry Collaboration" and automatically flagged as requiring Lecturer/Lab Manager approval.
SRS-13	The system shall authenticate registered users before allowing access to protected functions.
SRS-14	The system shall reject invalid login credentials.
SRS-15	The system shall provide role-based access according to the authenticated user's role.
SRS-16	The system shall allow guest users to submit a lab-visit request without authentication.

9. Software Architecture

Based on the SRS for the Lab Workflow & Request Management System (ISANPOWER888, Iteration 1 scope: request submission, tracking, staff processing, approval, and login), the recommended architecture is a **Modular Monolithic Architecture** using the **Model-View-Controller (MVC) pattern**. This is a single deployable application, internally organised into clearly separated layers, rather than a flat monolith or a distributed microservices system.

Rationale

The system is being built and maintained by a small student development team, and the current scope covers multiple related but bounded use cases: lab members submitting requests (UC-1), TAs viewing and processing the request queue (UC-2, UC-3, UC-4, UC-6), Lecturer/Lab Manager approval (UC-8), external visitor/collaborator requests via

public form or staff entry (UC-13, UC-14, UC-15), and user authentication (Login). A full microservices split would introduce unnecessary operational overhead — service discovery, inter-service networking, and multiple independent deployments — that a small team cannot realistically support at this stage, and the SRS does not require independent scaling of individual features.

A flat, undifferentiated monolith, on the other hand, risks becoming difficult to maintain as request-validation rules (SRS-4, SRS-5), authorisation and approval logic (SRS-3, SRS-7), and role-based access control (SRS-15) grow more complex across future iterations (notifications, booking/calendar integration, and analytics reporting are already flagged as later scope). The MVC modular approach gives the simplicity of a single deployment while cleanly separating presentation (View), request handling and authorisation (Controller), and business/data logic (Model), which keeps the codebase maintainable for the team and leaves the door open to extracting a module (e.g., the external-visitor/tracking-code subsystem, or notifications) into a separate service later if the system grows.

Component	Layer/Type	Description
Web Client	Presentation (View)	Renders the request submission form, request queue, request-detail/status views, the public visitor form, the tracking-code lookup page, and the login page; consumes data from the Controller and displays confirmations, validation errors, and status updates.
Request Controller	Controller	Receives HTTP requests (submit request, view queue, assign/update status, close request, approve/reject, submit visitor request, track by code), performs authorisation and approval checks (SRS-3, SRS-7), and coordinates calls to the Model layer before returning a response to the View.

Auth Controller	Controller	Handles login requests, verifies credentials, creates the user session, and returns role information used for role-based access (SRS-13, SRS-14, SRS-15).
Request Model / Business Logic	Model	Encapsulates request business rules: validates required fields (SRS-4, SRS-5), manages status transitions including Pending/Assigned/In Progress/Approved/Rejected/Closed (SRS-6), applies the approval-required flow (SRS-7, SRS-12), generates and validates tracking codes for external requests (SRS-9, SRS-10, SRS-11), and applies the active-queue/history split on close (SRS-1, SRS-8).
User Model	Model	Encapsulates user accounts, roles, and permissions; verifies credentials against stored records for the Auth Controller.
Request Database	Persistence	Stores request records (type, description, priority, requester, status, timestamps, approval info), request status-change history, user accounts/roles, and external-request tracking codes; serves queries for the active queue, individual request history, and status lookup by code.

Authentication/Authorisation Module	Cross-cutting	Verifies that only authorised staff (TA/Lab Manager) can assign, update, or close requests, and that only the Lecturer/Lab Manager can approve or reject approval-required requests (SRS-3, SRS-7); enforces role-based access (SRS-15) used by both Controllers before any protected operation.

10. Data Storage Technology

The recommended data storage technology is a Relational Database Management System (RDBMS), such as PostgreSQL or MySQL, used as the single data store for this iteration.

Rationale

All data described in the SRS is structured and fits a fixed, well-defined schema: users have roles and permissions, requests have a type, description, priority, requester, status, and timestamps, while some requests also require approval and maintain a traceable status history. The system also needs to maintain relationships between users and their submitted requests, authorised staff and their handled requests, and requests and their approval or status-change records.

There are no nested, variable-shape, or rapidly evolving data structures in this iteration that would justify a document-oriented or key-value NoSQL store. An RDBMS additionally enforces data integrity constraints and supports transactional consistency, which directly supports the requirements for validating required request information, maintaining valid request statuses, recording approval decisions, and preserving the history of status changes. The relational model also makes it straightforward to query and filter requests by type, priority, status, requester, and other structured fields.

Given the structured nature of the system and the clear relationships between its main entities, a relational database keeps the technology stack simple to operate and query while providing the consistency required for request tracking and approval workflows.

Component	Data Storage Technology	Description
User and Request Database	RDBMS (e.g., PostgreSQL/MySQL)	Relational database storing user accounts, roles, permissions, request records, request types, priorities, statuses, approval information, and timestamps.
Request History	RDBMS (e.g., PostgreSQL/MySQL)	Stores request status changes and timestamps to maintain a traceable history of each request.

Only one data store is required for the current SRS scope, since the main entities of the system are structured and can be managed through the same relational database. If future iterations introduce less-structured data or additional system features, supplementary storage technologies could be added without replacing the relational database used for the core request-management data.

11. Development Environment

The recommended development and deployment environment is a **Virtual Machine (VM) / containerised environment** , rather than bare metal.

Rationale

The Lab Workflow & Request Management System consists of an application and a relational database that need to operate consistently during development and deployment. A VM or containerised environment provides an isolated and reproducible environment for the application and its RDBMS.

A containerised environment allows the application and database to be configured separately while still working together as one system. This makes the development environment easier to reproduce across different machines and reduces problems caused by differences in software configurations or dependencies.

Compared with bare-metal development, a VM or containerised environment also provides better isolation. The development team can test changes to the application and database without directly affecting the host environment. The same environment

can then be reused during deployment, making the system easier to set up and maintain.

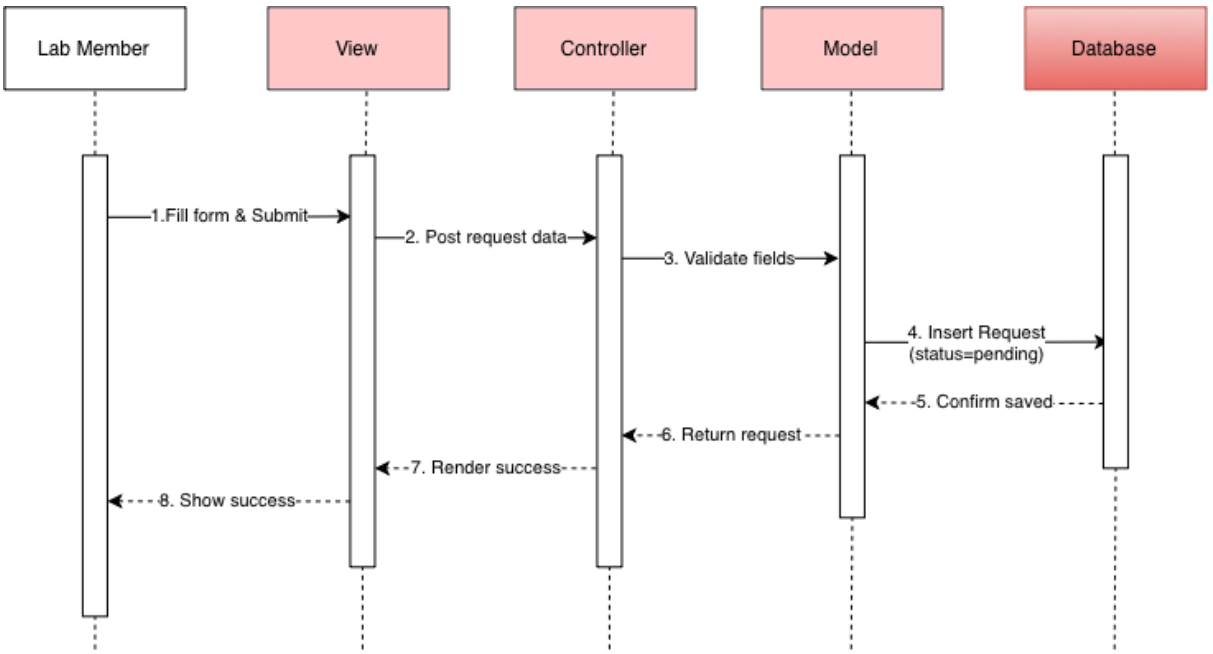
Component	Development Environment	Description
Application	Docker container	Runs the application and its business logic in an isolated and reproducible environment.
Request Database (RDBMS)	Docker container (via Docker Compose)	Runs the relational database used to store users, requests, statuses, approvals, and request history.
Development Machine	Virtual Machine or local Docker Desktop	Provides the environment used by the development team to build and test the system.

A VM or containerised environment is therefore suitable for the current system because it provides isolation and repeatability while making the application and database easier to develop, test, and deploy.

12. Sequence Diagrams for All SRS

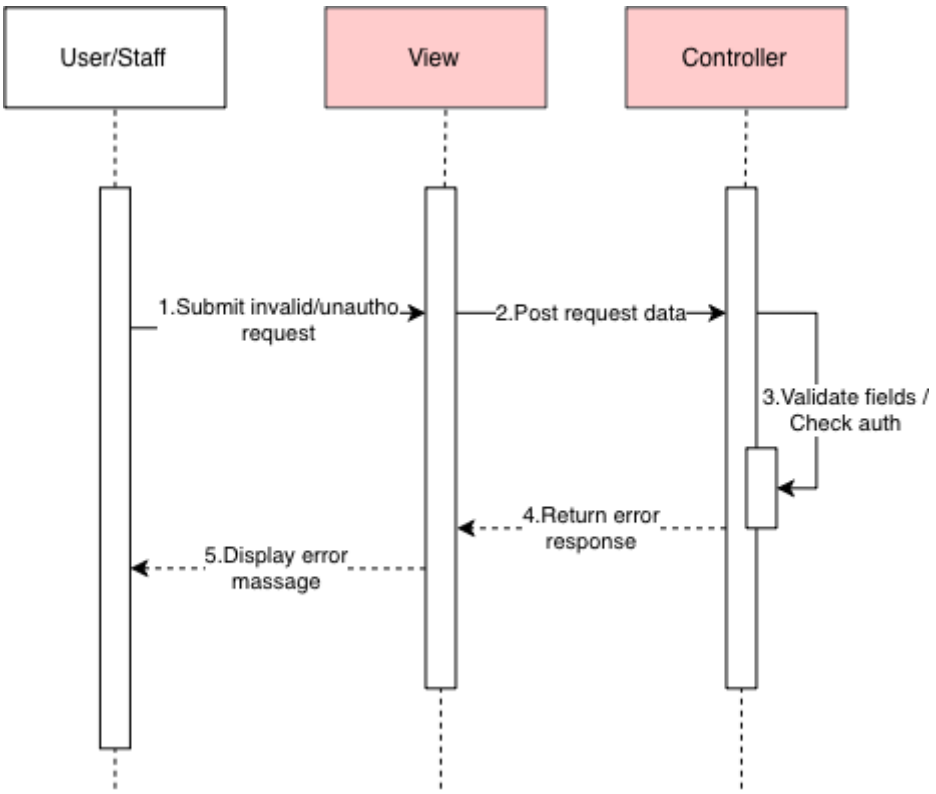
The sequence diagrams are derived from the Modular/MVC architecture (View, Controller, Model, Database) and cover all user workflows across SRS-1 to SRS-16

SQD-1: Lab Member Submits Request
Traced requirements: SRS-4, SRS-6



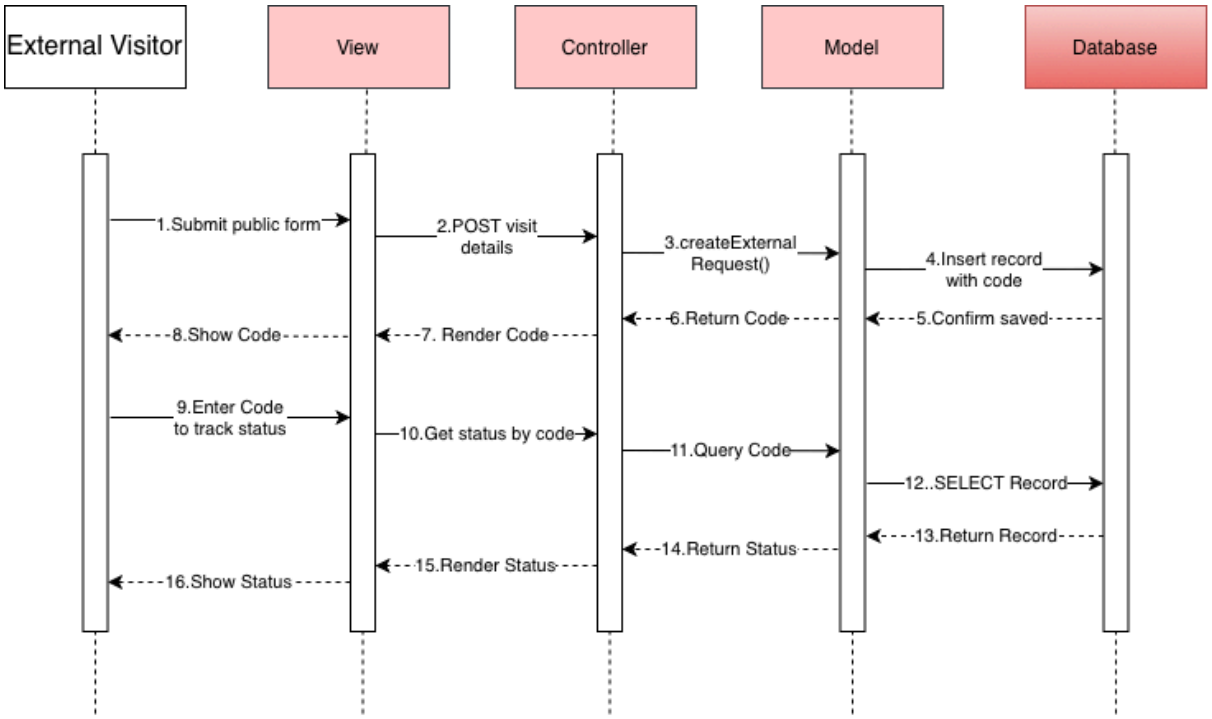
SQD-2: Request Submission / Authorization Error Path

Traced requirements: SRS-3, SRS-5



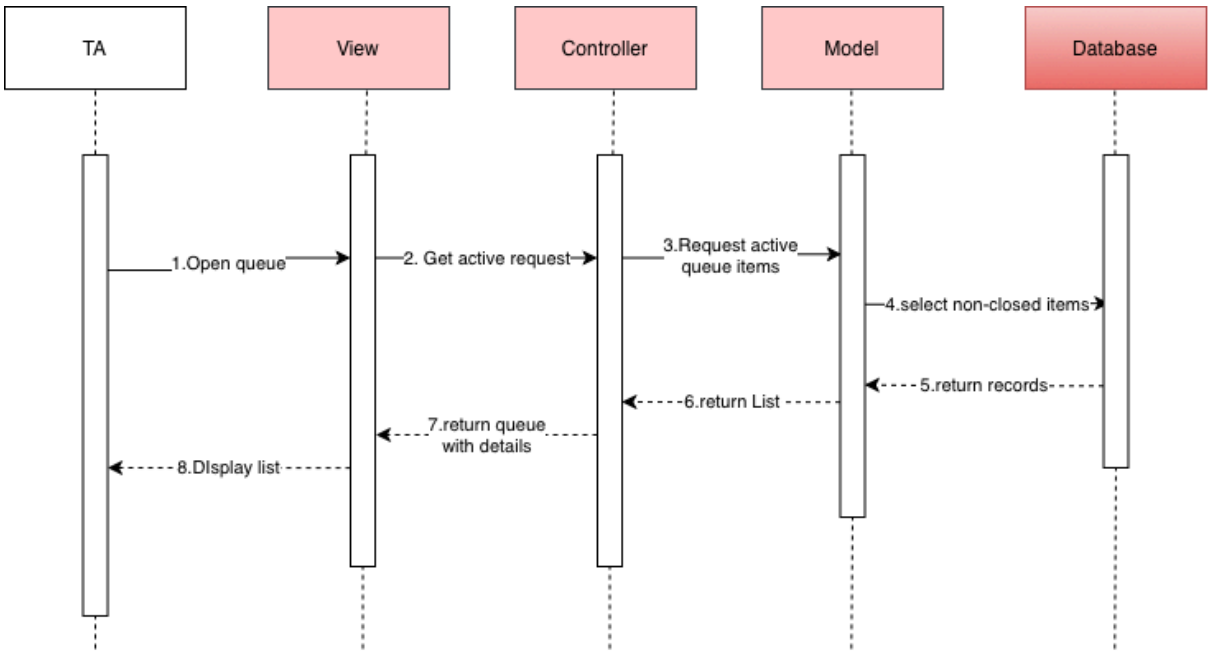
SQD-3: External Visitor Submits Visit Request & Tracks Status

Traced requirements: SRS-9, SRS-10, SRS-11



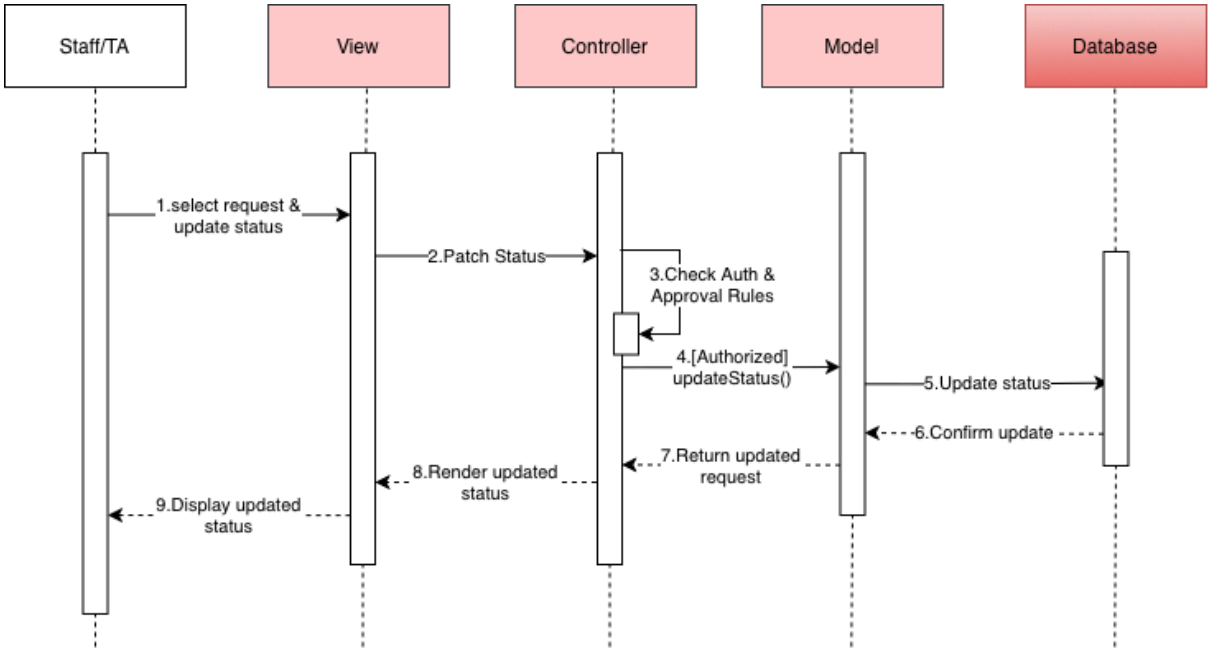
SQD-4: Teaching Assistant (TA) Views Active Request Queue

Traced requirements: SRS-1, SRS-2



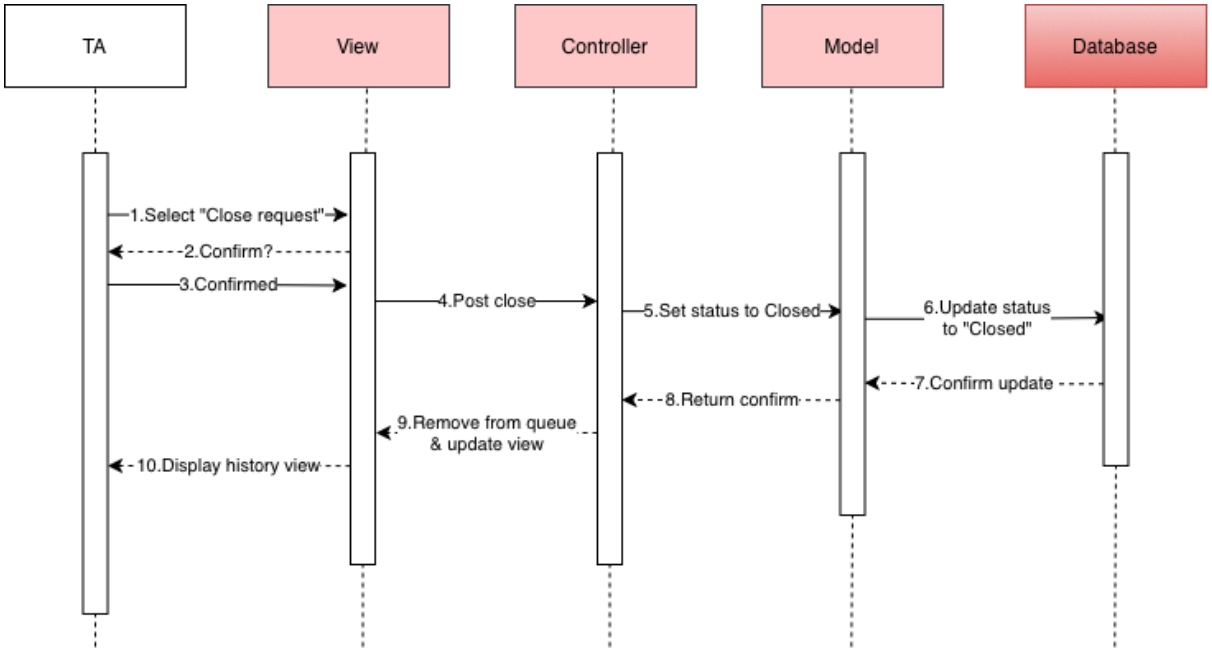
SQD-5: Authorised Staff Updates Request Status & Approval Path

Traced requirements: SRS-3, SRS-7, SRS-12



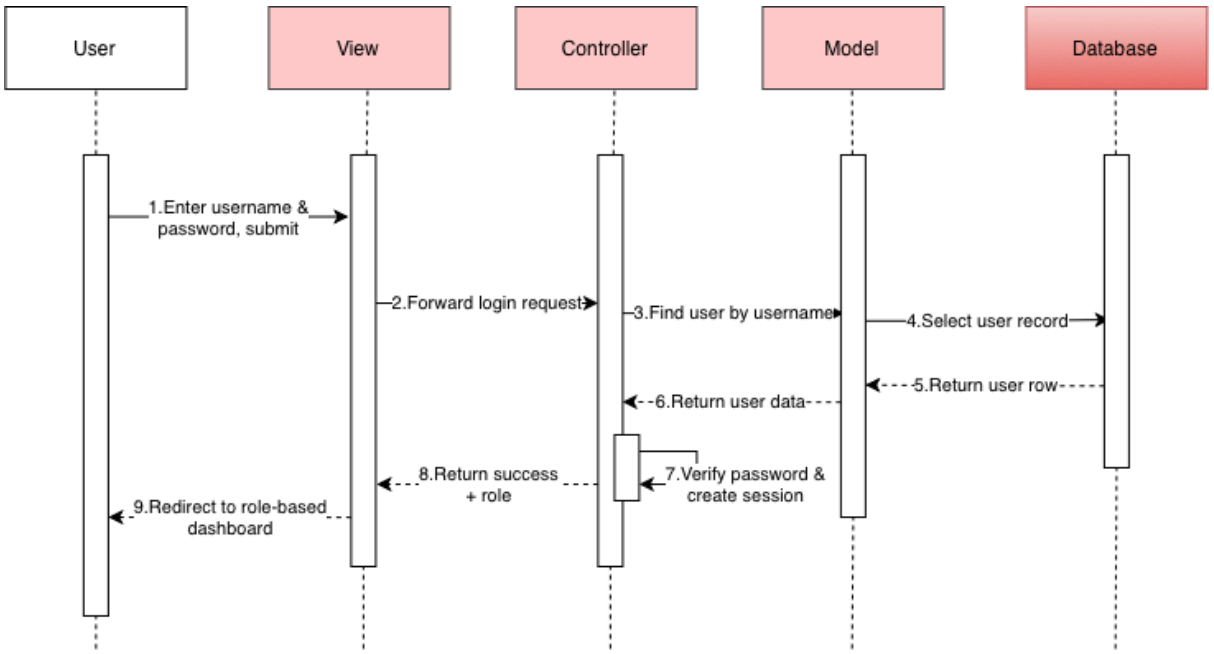
SQD-6: TA Closes Request (Archiving)

Traced requirements: SRS-3, SRS-6, SRS-8



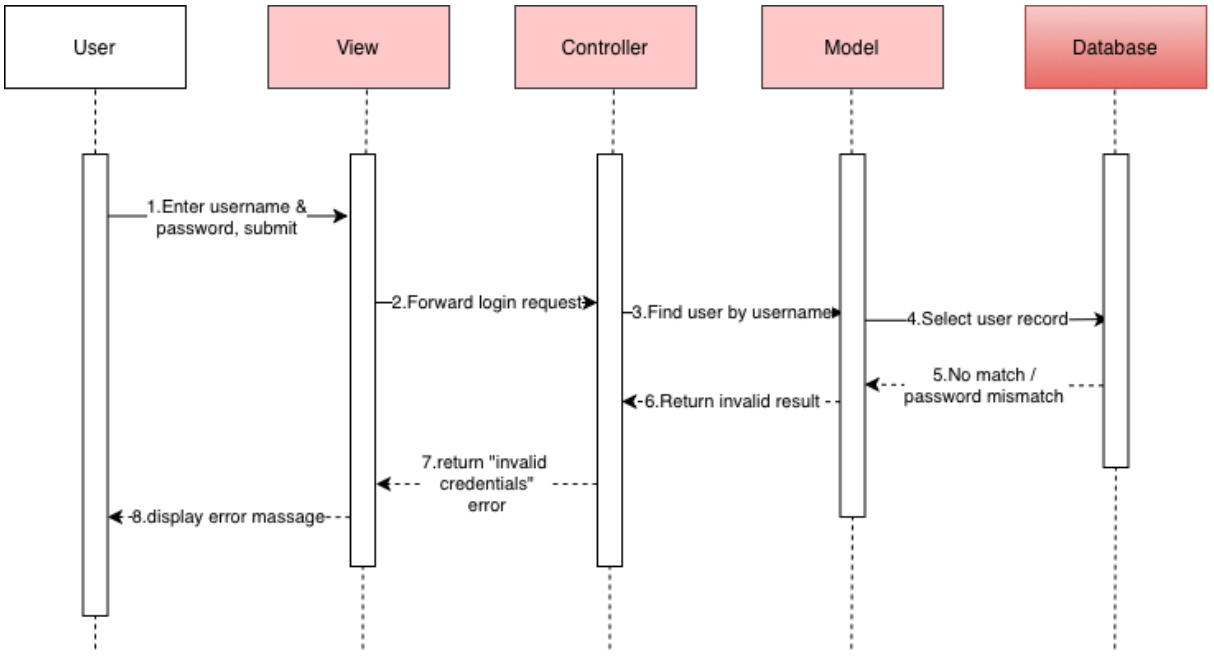
SQD-7: User Login (Success)

Traced requirements: SRS-13



SQD-8: User Login (Invalid Credentials)

Traced requirements: SRS-13



13. Traceability Matrix

From Sub-Goal to SRS

Sub-Goal	Use Case	User Story	URS	Activity Diagram	SRS
SG-1	UC-1 Submit Request	US-1	URS-1	AD-1	SRS-3, SRS-4
SG-2	UC-1 Submit Request, UC-2	US-1, US-3	URS-1, URS-3	AD-1, AD-2	SRS-2, SRS-4, SRS-5

	View Request Queue				
SG-3	UC-3 Assign Request, UC-4 Update Request Status	US-4	URS-4	AD-3	SRS-3, SRS-6, SRS-7
SG-4	UC-2 View Request Queue, UC-5 View My Requests	US-2, US-3	URS-2, URS-3	AD-2	SRS-1, SRS-2
SG-5	UC-6 Close Request, UC-7 View Dashboard/Reports	US-6	URS-6	AD-4	SRS-8
SG-6	UC-14 Submit Visit Request (Public), UC-13 Submit on Behalf of Collaborator, UC-15 Track Request	US-7, US-8	URS-8, URS-9, URS-10	AD-5	SRS-9, SRS-10, SRS-11, SRS-12
SG-7	UC-16 Login	US-9	URS-11, URS-13	AD-6	SRS-13, SRS-14, SRS-15

From SRS to Sequence Diagram

SRS	SQD
SRS-1	SQD-4
SRS-2	SQD-4
SRS-3	SQD-2, SQD-5, SQD-6
SRS-4	SQD-1
SRS-5	SQD-2
SRS-6	SQD-1, SQD-6
SRS-7	SQD-5

SRS-8	SQD-6
SRS-9	SQD-3
SRS-10	SQD-3
SRS-11	SQD-3
SRS-12	SQD-5
SRS-13	SQD-7, SQD-8
SRS-14	SQD-8
SRS-15	SQD-7
SRS-16	SQD-3